

SOFTWARE REQUIREMENTS SPECIFICATION

Менеджер задач с автосохранением

Версия документа: 1.0

05.05.2026

1. Введение

Продукт представляет собой оконное приложение для операционной системы Linux, разработанное на языке C++ с использованием фреймворка Qt. Приложение позволяет пользователю создавать, просматривать, редактировать и удалять задачи. Все данные автоматически сохраняются в фоновом режиме в локальные файлы без блокировки пользовательского интерфейса.

В состав программного продукта входят:

- Графический интерфейс пользователя (GUI);
- Модуль управления списком задач в памяти (TaskManager);
- Модуль фонового сохранения данных (Save), работающий в отдельном потоке;
- Модуль хранения задач и черновика в текстовых файлах (имитация базы данных).

Целевая аудитория:

- В рамках курсовой работы - студенты и преподаватели;
- Пользователи, которым требуется лёгкое приложение для ведения списка дел с гарантией сохранности данных.

Приложение запускается на ПК под управлением Linux. Пользователь взаимодействует с интерфейсом. Данные сохраняются в файлы tasks.txt (список задач) и draft.txt (черновик ввода).

2. Функциональные требования

2.1. Use Cases

UC-01 | Добавить новую задачу

Пользователь вводит текст задачи и нажимает кнопку «Add task». Задача добавляется в список, отображается с временной меткой, сохраняется в файл.

UC-02 | Редактировать существующую задачу

Пользователь выбирает задачу, нажимает «Edit task», изменяет текст и сохраняет изменения (кнопка «Save changes»). Обновляется временная метка задачи.

UC-03 | Удалить задачу

Пользователь выбирает задачу и нажимает «Delete task». Задача удаляется из списка и из файла.

UC-04 | Просмотр списка задач

Все задачи отображаются в списке с указанием даты последнего изменения.

UC-05 | Автосохранение черновика

Текст, введённый в поле ввода, автоматически сохраняется в draft.txt каждые 2 секунды без участия пользователя.

UC-06 | Восстановление данных при запуске

При старте программы список задач загружается из tasks.txt, а поле ввода заполняется последним сохранённым черновиком из draft.txt.

2.2. Бизнес-процессы

UC-01: Добавление новой задачи

В основном потоке:

- 1) пользователь вводит текст в поле lineEditTask;

- 2) нажимает кнопку `pushButtonAdd`;
- 3) приложение создаёт объект `Task` с текущей временной меткой;
- 4) задача добавляется в `std::vector<Task>` через `TaskManager::addTask()`;
- 5) список задач в GUI обновляется;
- 6) отправляется сигнал `requestSave` в поток сохранения;
- 7) поле ввода очищается.

В потоке сохранения:

- 1) при получении сигнала `requestSave` задача сохраняется в файл `tasks.txt`, а черновик как пустая строка - в `draft.txt`.

UC-02: Редактирование задачи

- 1) пользователь выбирает задачу в `listWidget`;
- 2) нажимает кнопку `pushButtonEdit`;
- 3) текст задачи подставляется в поле ввода, кнопка "Add task" меняет текст на "Save changes";
- 4) кнопки "Edit task" и "Delete task" отключаются и скрывают свой текст;
- 5) пользователь изменяет текст и нажимает "Save changes";
- 6) приложение обновляет описание задачи и временную метку;
- 7) интерфейс возвращается в обычный режим;
- 8) список задач в GUI обновляется и сохраняется в файл в потоке сохранения.

UC-03: Удаление задачи

- 1) пользователь выбирает задачу и нажимает "Delete task";
- 2) задача удаляется из вектора, интерфейс обновляется, отправляется сигнал в поток сохранения.

UC-05: Автосохранение черновика

- 1) запускается таймер `m_draftTimer` с интервалом 2000 мс;
- 2) по истечении таймера вызывается слот `onSaveDraftTimer()`, который отправляет сигнал `requestSaveDraft` с содержимым поля ввода;
- 3) черновик из поля ввода сохраняется в потоке сохранения.

2.3. Модель данных

Класс Task

Атрибут: description

Тип: std::string

Описание: Текст задачи

Атрибут: timeStamp

Тип: std::chrono::system_clock::time_point

Описание: Дата и время последнего изменения

Методы

- getDescription(), setDescription(), getTimeStamp()
- toLine() – возвращает строку вида "дд.мм.гггг чч:мм:сс | описание"
- fromLine(line) – статический метод для восстановления задачи из строки
- updateDescription(newDesc) – обновляет описание и ставит текущее время

Класс TaskManager

Атрибут: tasks

Тип: std::vector<Task>

Описание: Хранилище задач в памяти

Атрибут: filename

Тип: std::string

Описание: Имя файла БД (tasks.txt)

Методы

- loadFromFile() - загружает задачи из текстового файла

- addTask(), removeTask(), editTask() - управление списком
- getTasks() - возвращает копию вектора для сохранения
- clear() - очищает вектор

Файл "tasks.txt"

Каждая задача записывается в отдельную строку в формате "(дд.мм.гггг чч:мм:сс) описание".

Файл "draft.txt"

Одна строка – текущее содержимое поля ввода (может быть пустой).

3. Нефункциональные требования

3.1. Производительность

- Автосохранение: Фоновое сохранение не должно блокировать интерфейс. Для этого операция записи файлов выполняется в отдельном потоке.
- Загрузка при старте: Чтение файлов должно происходить до отображения окна.

3.2. Надёжность и устойчивость

- Целостность данных: При аварийном завершении программы сохранённые данные не теряются. Потеря возможна только для изменений строки ввода, сделанных менее чем за 2 секунды до сбоя.
- Восстановление: При повторном запуске программа загружает последнее сохранённое состояние.
- Обработка ошибок файловой системы: В текущей реализации ошибки игнорируются. Рекомендуется добавить логирование, программа не должна падать при отсутствии файлов.

3.3. Безопасность

- Приложение не работает с сетью, не требует аутентификации.

- Доступ к файлам данных ограничен правами текущего пользователя ОС.

3.4. Аппаратные и программные требования

- ОС: Linux (тестировалось на Ubuntu 24.04)
- Библиотеки: Qt Framework (Qt Creator 13.0.0)
- Ресурсы: 64 МБ ОЗУ, 10 МБ дискового пространства

3.5. Многопоточность

- Поток пользовательского интерфейса обрабатывает сигналы и обновление виджетов.
- Отдельный поток QThread выполняет сохранение задач.
- Взаимодействие между потоками осуществляется через сигналы/слоты Qt с автоматической очередью сообщений.
- Доступ к данным: копия вектора задач передаётся по значению, что исключает гонки данных. Поток сохранения работает с копией, в то время как главный поток может продолжать модификацию оригинала.

3.6. Расширяемость

- Код разделён на логические модули (UI, логика, сохранение).
- При необходимости можно заменить файловое хранилище на SQLite, изменив только класс Save.

4. Глоссарий

- Задача (Task) - Элемент списка дел, имеет текстовое описание и временную метку.
- Черновик (Draft) - Текст, введённый пользователем в поле ввода, но ещё не добавленный как задача. Сохраняется автоматически.
- Автосохранение - Автоматическая запись данных в файл без явной команды пользователя, происходит при каждом изменении.
- Таймер черновика - QTimer с интервалом 2 секунды, инициирующий сохранение черновика.

- Фоновый поток - Отдельный QThread, используемый для выполнения операций сохранения без блокировки главного потока.